

Preorder Root→Left→Right (parent first)	Inorder Left→Root→Right (sorted in BST)	Postorder Left→Right→Root (children first)	BFS Level by level (Queue)
--	--	---	-----------------------------------

1.1 TREE VS BINARY TREE VS BINARY SEARCH TREE (BST)

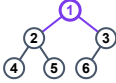
TREE

- A hierarchical structure
- Each node can have 0 or more children
- No ordering between child nodes



BINARY TREE

- Each node has at most 2 children: Left and Right
- No ordering between left and right subtree



BINARY SEARCH TREE (BST)

- Binary Tree + Left < Root < Right for every node
- Inorder traversal gives sorted order



1.3 KEY TERMINOLOGY

Node	Basic unit of a tree	Root	Topmost node
Edge	Connection between nodes	Parent	Immediate ancestor
Child	Immediate descendant	Leaf Node	Node with no children
Internal Node	Node with ≥1 child	Siblings	Children of same parent
Depth of Node	Edges from root to node (root=0)	Level of Node	Depth + 1 (root level=1)
Height of Node	Longest path from node to leaf	Height of Tree	Height of root node
Subtree	A node and all its descendants	Path	Sequence of nodes via edges

1.2 TYPES OF BINARY TREES

Full Binary Tree
Every node has 0 or 2 children.

Complete Binary Tree
All levels filled except possibly last; last level filled left to right.

Perfect Binary Tree
All internal nodes have 2 children; all leaves at same level.

Balanced Binary Tree
For every node: |height(left) – height(right)| ≤ 1

Skewed Binary Tree
Every node has only one child. (Left or Right Skewed)

2. TRAVERSALS — WAYS TO VISIT EVERY NODE OF A TREE

2.1 Depth First Search (DFS) Traversals

1 PREORDERRoot→Left→Right

Order: 1 - 2 - 4 - 5 - 3 - 6 - 7
AHA: Need parent first. Visit root before its children.
Use for: Build Tree, Serialize/Clone Tree, Prefix expressions

2 INORDERLeft→Root→Right

Order: 4 - 2 - 5 - 1 - 6 - 3 - 7
AHA: In BST, Inorder gives sorted order!
Use for: BST Problems, Kth Smallest/Largest, Validate BST

3 POSTORDERLeft→Right→Root

Order: 4 - 5 - 2 - 6 - 7 - 3 - 1
AHA: Need children's answers first. Process children before parent.
Use for: Height, Diameter, Balanced Tree, LCA, Path Sum

2.2 BFS — LEVEL ORDER TRAVERSAL

Order: 1 - 2 - 3 - 4 - 5 - 6 - 7
AHA: Need level by level. Use Queue. Queue size tells current level.

```
Queue<TreeNode> q = new LinkedList<>();
q.offer(root);
while (!q.isEmpty()) {
    int size = q.size(); // current Level
    List<Integer> level = new ArrayList<>();
    for (int i = 0; i < size; i++) {
        TreeNode node = q.poll();
        level.add(node.val);
        if (node.left != null) q.offer(node.left);
        if (node.right != null) q.offer(node.right);
    }
    res.add(level);
}
```

2.3 DFS VS BFS COMPARISON

Feature	DFS	BFS
Strategy	Go deep, then backtrack	Level by level
Data Structure	Stack / Recursion	Queue
Time	O(n)	O(n)
Space	O(h) — height	O(w) — max width
Best For	Path, Height, DP on trees, Backtracking	Level-wise problems, Shortest path (unweighted)

2.4 WHEN TO USE WHICH TRAVERSAL?

Need parent before children? → **Preorder**

Need sorted order (BST)? → **Inorder**

Need children's results first? → **Postorder**

Need level by level? → **BFS (Level Order)**

Modify / Build / Serialize? → **Mostly Preorder**

★ **FINAL MANTRA:** Identify Pattern → Apply Correct Traversal → Write Clean Code → Dry Run → Verify

Next: Recursion Template + DFS Patterns →

TREES – RECURSION TEMPLATE & DFS PATTERNS

Faith → Recur → Combine → Return · 5 DFS Pattern Problems (Postorder Focused)

Postorder → Children's answers first = Height, Diameter, Balanced, LCA, Path Sum	Preorder → Parent first = Invert, Serialize, Build Tree	FAITH → Assume recursive calls give correct answers
---	--	--

3. TREE RECURSION TEMPLATE (FAITH → RECUR → COMBINE → RETURN)

3.1 The Recursion Flow

1

BASE CASE — Check and handle the base condition.
if (node == null) return ...;

↓

2

FAITH — Assume left and right subtrees will give correct answers.

↓

3

RECUR LEFT — Recur for the left subtree.
left = solve(node.left);

↓

4

RECUR RIGHT — Recur for the right subtree.
right = solve(node.right);

↓

5

COMBINE — Use left and right results. Do the required work.
ans = ...;

↓

6

RETURN — Return the final answer up the stack.
return ans;

Universal Template

```
Type solve(Node node) {
    if (node == null) {
        return ...; // Base Case
    }
    Type left = solve(node.left); // Recur Left
    Type right = solve(node.right); // Recur Right

    Type ans = ...; // Combine
    return ans; // Return
}
```

3.2 Return Type Decides Recursion

Return Type	What it Means	Examples
int	Return some value (height, count, sum)	Max Depth, Diameter
boolean	Return true/false info	Same Tree, Balanced
TreeNode	Return a node	LCA, Invert Tree
List/Vector	Return list of values/nodes	Level Order, All Paths

★ **FAITH** → You assume your recursive calls (left & right) are correct and return the right answer. Trust the recursion!

4. DFS Pattern Problems (Postorder Focused) — Most important tree problems follow these patterns!

P1 MAX DEPTH OF BINARY TREE (LC 104)				
AHA MOMENT Need heights of left & right subtrees first, then take max. That's Postorder!	PATTERN Postorder (Left→Right→Root) Formula: 1 + max(left, right)	COMPLEXITY Time: O(n) Space: O(h) h = height	JAVA CODE <pre>int maxDepth(TreeNode root) { if (root == null) return 0; // Base Case int left = maxDepth(root.left); // Recur Left int right = maxDepth(root.right); // Recur Right return 1 + Math.max(left, right); // Combine }</pre>	EXAMPLE Tree with 3 levels → Depth = 3
P2 SAME TREE (LC 100)				
AHA MOMENT Both trees must match at current node, left subtree and right subtree. Return boolean.	PATTERN Postorder Formula: left && right Base Case: Both null → true One null → false	COMPLEXITY Time: O(n) Space: O(h)	JAVA CODE <pre>boolean isSameTree(TreeNode p, TreeNode q) { if (p == null && q == null) return true; if (p == null q == null) return false; if (p.val != q.val) return false; boolean left = isSameTree(p.left, q.left); boolean right = isSameTree(p.right, q.right); return left && right; // Combine }</pre>	USE FOR LC 100 Same Tree LC 572 Subtree of Another Tree
P3 INVERT BINARY TREE (LC 226)				
AHA MOMENT Need to process parent first, then swap children and recurse. That's Preorder!	PATTERN Preorder (Root→Left→Right) Formula: Swap Left and Right	COMPLEXITY Time: O(n) Space: O(h)	JAVA CODE <pre>TreeNode invertTree(TreeNode root) { if (root == null) return null; TreeNode temp = root.left; // Swap root.left = root.right; root.right = temp; invertTree(root.left); // Left invertTree(root.right); // Right return root; }</pre>	USE FOR Mirror tree problems
P4 DIAMETER OF BINARY TREE (LC 543)				
AHA MOMENT Diameter can pass through any node. Need left height + right height at every node. Update global answer!	PATTERN Postorder Formula: diameter = max(diameter, left+right) return height = 1 + max(left, right)	COMPLEXITY Time: O(n) Space: O(h)	JAVA CODE <pre>int diameter = 0; int diameter(TreeNode root) { height(root); return diameter; } int height(TreeNode node) { if (node == null) return 0; int left = height(node.left); int right = height(node.right); diameter = Math.max(diameter, left+right); }</pre>	KEY NOTE LC 543 Diameter of Binary Tree. Global variable updated during traversal.

			<pre>return 1 + Math.max(left, right); }</pre>	
--	--	--	--	--

P5 BALANCED BINARY TREE (LC 110)				
AHA MOMENT Every node must be balanced. Not just the root. Check $left-right \leq 1$ at every node using height helper.	PATTERN Postorder Formula: if $ left-right > 1$: return -1 else return $1+\max(l,r)$	COMPLEXITY Time: $O(n)$ Space: $O(h)$	JAVA CODE <pre>boolean isBalanced(TreeNode root) { return height(root) != -1; } int height(TreeNode node) { if (node == null) return 0; int left = height(node.left); if (left == -1) return -1; int right = height(node.right); if (right == -1) return -1; if (Math.abs(left-right) > 1) return -1; return 1 + Math.max(left, right); }</pre>	KEY NOTE Returns -1 on imbalance. Propagates up immediately without extra traversal.

POSTORDER QUICK RECAP — WHEN TO USE POSTORDER			
Height/Depth $1 + \max(\text{left}, \text{right})$	Diameter $\text{left} + \text{right}$ (update ans)	Balanced $\text{abs}(\text{left} - \text{right}) \leq 1$	Same Tree $\text{left} \&\& \text{right}$
Use Postorder when: Need children's answers first · Need height/depth/size · Need info from left & right to compute current node · Delete/Free/Evaluate expressions · Diameter, Balanced, Max Path Sum, LCA (Binary Tree) etc.			

★ FINAL MANTRA: Identify Pattern → Write Template → Apply Formula → Handle Base Cases → Return Correct Type	Next: BFS Patterns + BST Bible →
--	---

BFS = Level by Level using Queue **BST Rule:** Left Subtree < Node < Right Subtree (for every node) **Inorder of BST** = Sorted Order (your best friend!)

5. BFS Pattern Problems (Level Order Focused)

P1 LEVEL ORDER TRAVERSAL (LC 102)

AHA

Need nodes level by level. Use Queue. queue.size() = current level size. Process all nodes of a level together.

PATTERN & IDEA

BFS (Level Order)

Process all nodes in queue (size = current level). Add children for next level.

Time: O(n) **Space:** O(n)

JAVA CODE

```
List<List<Integer>> res = new ArrayList<>();
Queue<TreeNode> q = new LinkedList<>();
q.offer(root);
while (!q.isEmpty()) {
    int size = q.size();
    List<Integer> level = new ArrayList<>();
    for (int i = 0; i < size; i++) {
        TreeNode node = q.poll();
        level.add(node.val);
        if (node.left != null) q.offer(node.left);
        if (node.right != null) q.offer(node.right);
    }
    res.add(level);
}
return res;
```

ALSO USES

LC 107 Level Order II
LC 199 Right Side View
LC 513 Find Bottom Left

P2 RIGHT SIDE VIEW (LC 199)

AHA: We need the last node of each level. In each level, the last node we pop is the rightmost.

```
List<Integer> rightSideView(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size();
        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();
            if (i == size-1) res.add(node.val); // Last
            if (node.left != null) q.offer(node.left);
            if (node.right != null) q.offer(node.right);
        }
    }
    return res;
}
```

Time: O(n) **Space:** O(w)

P3 ZIGZAG LEVEL ORDER (LC 103)

AHA: Direction changes at every level. Use a flag (leftToRight). Toggle after each level. Use LinkedList to addFirst or addLast.

```
boolean leftToRight = true;
while (!q.isEmpty()) {
    int size = q.size();
    LinkedList<Integer> level = new LinkedList<>();
    for (int i = 0; i < size; i++) {
        TreeNode node = q.poll();
        if (leftToRight) level.addLast(node.val);
        else level.addFirst(node.val); // reverse
        if (node.left != null) q.offer(node.left);
        if (node.right != null) q.offer(node.right);
    }
    res.add(level);
    leftToRight = !leftToRight; // toggle
}
```

Time: O(n) **Space:** O(w)

6. BST Bible — Binary Search Tree (The most important BST rules, operations and patterns)

1. BST RULE

All values in Left Subtree < N < All values in Right Subtree

- Left subtree → smaller values
- Right subtree → larger values
- Inorder traversal gives sorted order

2. SEARCH IN BST

Use BST property to decide direction:

```
if key == root.val → found
elif key < root.val → go left
else → go right
```

Time: O(h) **Space:** O(1) iter / O(h) rec

7. LCA IN BST (LC 235)

Use BST property to find LCA:

```
if both p, q < root → go left
if both p, q > root → go right
else → root is LCA
```

Time: O(h) **Space:** O(1) iter / O(h) rec

5. KTH SMALLEST ELEMENT IN BST (LC 230)

Idea: Inorder traversal of BST gives sorted order.

Steps: 1. Do Inorder (Left → Root → Right) 2. Count nodes 3. When count == k → return node

```
int count = 0; int result = 0;
void inorder(TreeNode node, int k) {
    if (node == null) return;
    inorder(node.left, k);
    if (++count == k) { result = node.val; return; }
    inorder(node.right, k);
}
```

Time: O(h+k) **Space:** O(h)

6. VALIDATE BST (LC 98)

Idea: Each node must lie in a valid range. For every node: min < node.val < max

```
boolean isValid(TreeNode node, Long min, Long max) {
    if (node == null) return true;
    if (node.val <= min || node.val >= max) return false;
    return isValid(node.left, min, (long)node.val) &&
        isValid(node.right, (long)node.val, max);
}
```

Call: isValid(root, Long.MIN_VALUE, Long.MAX_VALUE)

Time: O(n) **Space:** O(h)

4. DELETE FROM BST (3 CASES)

Case 1: X is a leaf node → Just delete it.

Case 2: X has one child → Replace X with its child.

Case 3: X has two children → Replace X with Inorder Successor (smallest in right subtree) or Inorder Predecessor (largest in left subtree).

Time: O(h) **Space:** O(1) iter / O(h) rec

BST CHEAT SHEET

Operation	Idea	Time	Space
Search	BST property	O(h)	O(1)
Insert	Find & insert	O(h)	O(1)
Delete	3 cases	O(h)	O(h)
Kth Smallest	Inorder+count	O(h+k)	O(h)
Validate BST	Range (min,max)	O(n)	O(h)
LCA (BST)	BST property	O(h)	O(1)

REMEMBER: Inorder = Sorted · Use range for validation · BST gives direction

8. BST PROPERTIES + INORDER = SORTED ORDER

✓
Left < Root < Right

✓
No duplicates (by default)

✓
Balanced BST → $O(\log n)$

Why Inorder = Sorted?

All left < root < all right. So visiting Left → Root → Right gives increasing order.

✓
Inorder = Sorted

✓
Efficient: $O(h)$ operations

Think in terms of range. Inorder is your best friend. BST gives you direction.

BST: 1 → 3 → 6 → 8 → 10 → 14 (sorted!)

★ Understand Rule → Use Property → Apply Pattern → Inorder for Sorted → Range for Validation → Practice & Master

Next: Advanced Problems + AHA
Revision →

TREES – ADVANCED PROBLEMS & FAANG REVISION

LCA · Construct from Traversals · Path Sum · Serialize/Deserialize · AHA Moments · One-Page Revision

Preorder → Root comes first (Build/Serialize) **Inorder** → Left/Right split point (Construct) **Postorder** → Root comes last (Build from post+in) **Graph+BFS** → Burning Tree, Multi-source BFS

7. Advanced Tree Problems — Important Problems Beyond Basics

1. LCA in Binary Tree (LC 236)

AHA: Children report back. If both sides return non-null → current node is LCA.

Pattern: Postorder

```
if (root == null || root == p || root == q) return root;
TreeNode left = lca(root.left, p, q);
TreeNode right = lca(root.right, p, q);
if (left != null && right != null) return root;
return left != null ? left : right;
```

Time: O(n) **Space:** O(h)

2. Build Tree from Preorder + Inorder (LC 105)

AHA: Preorder → Root comes first. Inorder → Left and Right parts. Build tree using indices.

Pattern: Recursive (Divide & Conquer)

Idea / Formula:

- First element in preorder = Root
- Find root in inorder → split left & right
- Recur for left and right part

Time: O(n) **Space:** O(n)

3. Serialize and Deserialize Binary Tree (LC 297)

AHA: Store structure + values. Use null marker for missing nodes. BFS Level Order or Preorder.

Pattern: BFS (Level Order) or Preorder

Serialize: Traverse and add values (add nulls for missing)

Deserialize: Rebuild tree from saved values

Example: Tree [1,2,3,4,5] → Serialized: "1,2,4,null,null,5,3,null,6"

Time: O(n) **Space:** O(n)

4. Path Sum (LC 112 / 113)

AHA: Need root-to-leaf path. Keep subtracting from target. If leaf and target == 0 → valid path.

Pattern: DFS (Preorder)

```
boolean pathSum(TreeNode node, int target) {
    if (node == null) return false;
    target -= node.val;
    if (node.left == null && node.right == null) return target == 0;
    return pathSum(node.left, target) || pathSum(node.right, target);
}
```

Time: O(n) **Space:** O(h)

5. Build Tree from Inorder + Postorder (LC 106)

AHA: Postorder → Root comes last. Inorder → Left and Right parts. Build tree using indices.

Pattern: Recursive (Divide & Conquer)

Idea / Formula:

- Last element in postorder = Root
- Find root in inorder → split left & right
- Recur for left and right part

Time: O(n) **Space:** O(n)

6. Burning Binary Tree (LC 2385)

AHA: Fire spreads in all 4 directions. Convert tree to undirected graph. Do BFS from start node. Count levels (time).

Pattern: Graph + BFS

- Convert tree to undirected graph
- BFS from start node
- Count levels → time to burn all nodes

Time: O(n) **Space:** O(n)

QUICK RECAP — TREE PROBLEM MENTAL MODELS	
Need parent first	Preorder
Need sorted order	Inorder
Need children first	Postorder
Need level by level	BFS
Path / Root to Leaf	DFS (Backtrack)
Multiple directions	Graph + BFS
✓ Traversal decides the way you think.	
✓ Return type decides the recursion.	
✓ Draw the tree. Dry run. That's 80% of the battle!	

8. Trees — FAANG One-Page Revision (AHA Moments)

7. MENTAL MODELS — AHA MOMENTS (SAVE TO MEMORY)

★ **Preorder** → I do (parent) first

★ **Inorder** → I am in the middle (sorted for BST)

★ **Postorder** → I check results of my children first

★ **BFS** → I explore by level

★ **DFS** → I go deep and backtrack

★ **Max Depth** → 1 + max(left, right)

★ **Diameter** → left + right (at every node)

★ **Balanced** → abs(left - right) <= 1

★ **Invert Tree** → Swap + Recurse (Preorder)

★ **Every node can be center of diameter**

IMPORTANT REMINDERS: Draw the tree (always helps!) · Dry run before coding · Handle null carefully · Think recursively, not iteratively · Keep solving, keep improving!

8. COMPLEXITY CHEAT SHEET

Operation / Problem	Time	Space
Traversal (DFS)	O(n)	O(h)
Traversal (BFS)	O(n)	O(w)
Search in BST	O(h)	O(1)
Insert in BST	O(h)	O(1)
Delete in BST	O(h)	O(h)
Height of Tree	O(n)	O(h)
Diameter	O(n)	O(h)
LCA (Binary Tree)	O(n)	O(h)
LCA (BST)	O(h)	O(1)

n = nodes · h = height · w = max width
Balanced: h = O(log n) · Skewed: h = O(n)

9. DFS + BFS PATTERN RECAP

DFS Patterns		BFS Patterns	
Max Depth	1 + max(l,r)	Level Order	Queue + level size
Same Tree	left && right	Right Side View	Last node each level
Invert Tree	Swap l & r		

Diameter	Through every node	Zigzag Order	Left->Right toggle
Balanced	$ \text{left-right} \leq 1$	Average Levels	sum / size
		Level Order N-	Same BFS + N children

5. BST QUICK HITS + FINAL MANTRA

- ✓ Left Subtree < Node < Right Subtree
- ✓ Inorder of BST = Sorted Array
- ✓ Search / Insert = Compare & Go
- ✓ Delete = 3 cases (0 child, 1 child, 2 children)
- ✓ Validate BST = Check range (min, max)
- ✓ Kth Smallest = Inorder (kth element)
- ✓ LCA in BST = Use BST property
- ✓ Build Tree = Preorder root, Inorder split
- ✓ LCA in Binary Tree = Children report back

★ FINAL MANTRA: Understand Problem → Choose Pattern → Draw Tree → Apply Template → Code → Dry Run → Optimize

PRACTICE → ANALYZE → REFLECT → IMPROVE → REPEAT 🚀